

VisionInspect AI

Frontend Development — Real Backend Integration, Full Docker Containerization & Cloud Deployment

Milestone 4 — Individual Contribution Report

Ayusha

Infosys Springboard Virtual Internship Program

Table of Contents

Table of Contents.....	2
1. Project Description	3
2. Backend Integration Merge	3
3. Post-Merge Verification & Debugging.....	3
4. Docker Containerization.....	3
5. Cloud Deployment (Render).....	4
6. Interim Demo Access	4
7. Conclusion	5

1. Project Description

This report covers Milestone 4 of the frontend contribution to VisionInspect AI, an AI-powered manufacturing defect detection and quality inspection platform developed under the Infosys Springboard Virtual Internship Program (VIP). This milestone centered on three connected efforts: merging a major backend integration branch (real authentication, database-backed inspection history, and PDF reporting) into the frontend, fully containerizing the application with Docker, and deploying that containerized build to a cloud host to test real-world hosting behavior.

2. Backend Integration Merge

The team lead's branch introduced a substantial architecture change: real user accounts (JWT-based authentication), a relational database layer, per-user inspection history, an admin-only Factory Supervisor role, and PDF report generation — replacing the mock authentication and local-storage-based history used in earlier milestones.

Merging this branch into the frontend branch produced genuine conflicts (not silent auto-merges) in four files, each resolved individually:

- `components/InspectionTable.js`: combined both sides' changes — kept a null-safe search filter from the incoming branch and preserved the existing responsive button layout, adding the new PDF download action into it.
- `components/SupervisorOverview.js` and `components/InspectionResult.js`: the incoming branch's version was a complete superset of the existing code (same loading-state UI, plus real API calls, PDF export, and detailed inference data not previously present), so the incoming version was adopted in full rather than hand-merging line by line.
- `pages/dashboard.js`: combined the existing shared header component with the incoming branch's real backend-driven history and analytics fetching, removing the now-unnecessary local-storage read/write calls.

After resolving conflicts, the merge was verified with a production build before being trusted — and, consistent with the milestone's testing discipline, the app was also run and clicked through manually rather than relying on the build passing as proof of correctness.

3. Post-Merge Verification & Debugging

Manual testing after the merge surfaced two real issues that a passing build did not catch, both diagnosed and fixed directly:

- A runtime crash on the Factory Supervisor dashboard: the automatic merge had silently dropped a loading-state variable and its related component while keeping the code that referenced them, producing a "ReferenceError: isLoading is not defined" only visible when the page was actually opened. Fixed by restoring the loading state, correctly wired to the real API calls this time.
- A wrong backend entry point locally: the FastAPI server was being started from an older, disconnected file (`anomaly_detection/api.py`) instead of the actual integrated application (`backend/app.py`), which explained a 404 on the login endpoint even though the route existed and was correctly coded.

4. Docker Containerization

The full application — frontend, backend, and database — was containerized using Docker Compose as three coordinated services:

- Backend container: Python 3.12 with CPU-only PyTorch, OpenCV, Ultralytics YOLO, and a MySQL client library, running the FastAPI app via Uvicorn.
- Frontend container: a multi-stage Next.js build, with the backend API URL injected at build time as a build argument rather than hardcoded.
- Database container: MySQL 8, with a healthcheck so the backend only starts once the database is genuinely ready to accept connections, and a persistent volume so data survives container restarts.

Several real issues were diagnosed and resolved while getting this running end-to-end:

- A pip package hash-verification failure during the backend image build, caused by an unstable network connection corrupting a large download mid-transfer; resolved by adding retry and timeout flags to the pip install step.
- A host port conflict on 3306 (something else on the machine already using MySQL's default port); resolved by remapping the container's exposed port without changing the internal Docker-network port the backend actually connects to.
- A database compatibility issue: SQLite (used earlier in local testing) does not require a length on VARCHAR/String columns, but MySQL does. This surfaced as a `CompileError` on each table in turn as it was created, and was fixed by adding explicit lengths to every String column across six model files (users, inspection history, reports, user activity, batch inspection, inspection workflow, analytics).

5. Cloud Deployment (Render)

To test the Docker setup on real cloud infrastructure rather than only locally, the same Docker images were deployed to Render as two web services (backend and frontend) backed by a managed PostgreSQL database.

- The repository was forked to a personal GitHub account to work around not having admin access on the team's organization repository, which Render required to connect and deploy from.
- The database URL was adjusted for the PostgreSQL driver (`postgresql+psycopg2://`), requiring no code changes since the earlier column-length fixes made for MySQL compatibility carried over correctly to PostgreSQL as well.
- A frontend build failure caused by a missing `public/` directory (not tracked in git) was resolved by removing the now-unnecessary `COPY` instructions from the Dockerfile.
- After both services deployed successfully, CORS was updated on the backend to allow requests from the new Render frontend URL.
- Once live, running an actual inspection triggered a memory-limit crash and automatic restart on the backend — the same RAM constraint the backend/ML team had already encountered on their own deployment attempt, confirming it as a shared, infrastructure-level limitation of free-tier hosting rather than an issue specific to this deployment or this milestone's work.

6. Interim Demo Access

Since the cloud deployment's inference path is constrained by free-tier RAM (a backend/model-side fix, outside this milestone's scope), a locally-run Docker Compose stack — exposed via a reserved ngrok domain — was used as the working demo for team and mentor review in the meantime. The Render deployment link was documented in the project README as the permanent deployment target, with an honest note about its current memory limitation.

7. Conclusion

Milestone 4 moved the project from a locally-mocked frontend to one running against real authentication, a real database, and a real, cloud-deployed infrastructure. The merge was completed without silently losing either side's work, verified through actual use rather than a passing build alone, and the resulting Docker setup was proven correct on real cloud infrastructure — build, deploy, database connection, and CORS all working. The one remaining blocker, a memory limit during inference, is a known, team-wide constraint tied to model size rather than a defect in this milestone's integration or deployment work, and has been clearly handed off as a model-optimization task alongside a working interim demo link.